

计算机组成原理第四次实验报告

实验名称：ALU 操作扩展

学号：2410668

姓名：吕昊远

班次：张金、董前琨

一、实验目的

1. 熟悉 MIPS 指令集中的运算指令，学会对这些指令进行归纳分类。
2. 了解 MIPS 指令结构及设计原理。
3. 熟悉并掌握 ALU 的原理、功能和设计。
4. 深入理解 ALU 的设计，在实践中学习 ALU 操作扩展的设计流程。

二、实验内容

1. 阅读实验箱相关文档，学习 ALU 的基础功能与结构，将给出的代码配置为项目，生成位流文件烧录至实验箱进行基础功能的复现与验证。
2. 深入学习硬件中的 ALU 结构及其相关的计算逻辑与实现原理，并依此设计进行计算操作的扩展，绘制逻辑结构图。
3. 调整操作码的设置规则，由 12 位独热码（12 种运算）改为 4 位拨码开关（16 种运算）来控制，同时新增 4 种新指令的运算逻辑：
 - 判断相等 (SEQ)
 - 按位与非 (NAND)
 - 低位加载 (LLI)
 - 按位同或 (XNOR)
4. 根据设计的改进方案，使用 verilog 修改核心模块与显示模块以及引脚约束文件，并编写仿真文件进行仿真波形验证，完成后按照步骤生成位流文件，将其导入实验箱烧录，在实验箱上对比验证各项功能。

三、实验原理图

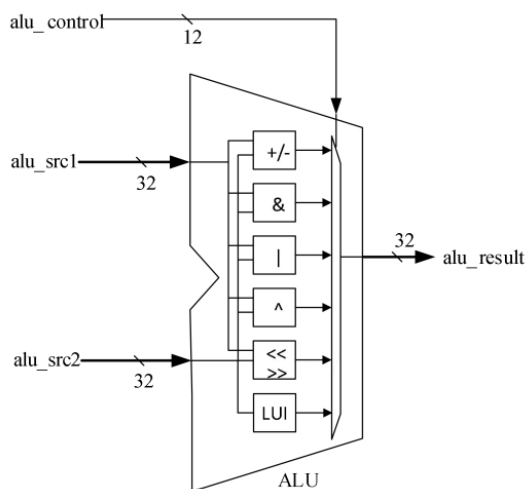


图 1: 基础 ALU 模块

依照指导书的原理图，我们可以知道，ALU 通过 12 位独热码控制逻辑运算的输出。在具体流程上，ALU 会先对两个输入的数完整地进行所需的全部运算，然后再根据控制码按需进行输出，从而得到结果。

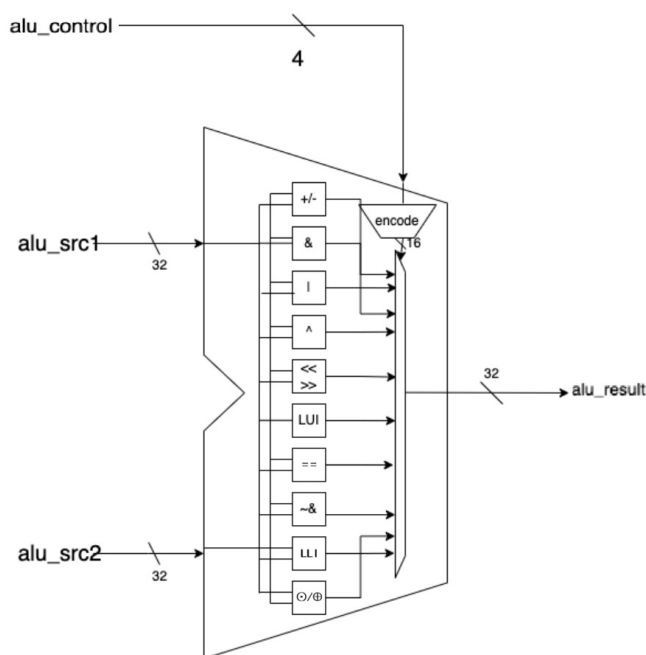


图 2: 改进后的 ALU 模块

在上述 ALU 模块的基础上继续进行功能改进：首先将 alu_control 改为 4 位，再通过新增的编码器将四位二进制数转为对应的 16 种控制码，使其可以在实验箱上通过 4 个拨码开关来控制运算。同时，运算由 12 种变为 16 种，还需扩展新增的 4 种运算，编写相应逻辑并配置好对应的控制码，使其可以正常输出。

四、实验过程

1. 基础 ALU 测试

使用 Vivado 创建项目文件,将老师提供的项目代码导入(为防止项目配置问题,此处使用文本级的复制粘贴进行导入),依据指导书完成设计文件与约束文件,检查代码无误后,依次运行 Synthesis 与 Implementation,最后生成位流文件 Bitstream;代码项目生成完毕后,连接实验箱,将位流文件烧录至开发板上,依照设计进行功能验证。

调节拨码开关,使用 LCD 屏触控输入,得到以下测试结果:

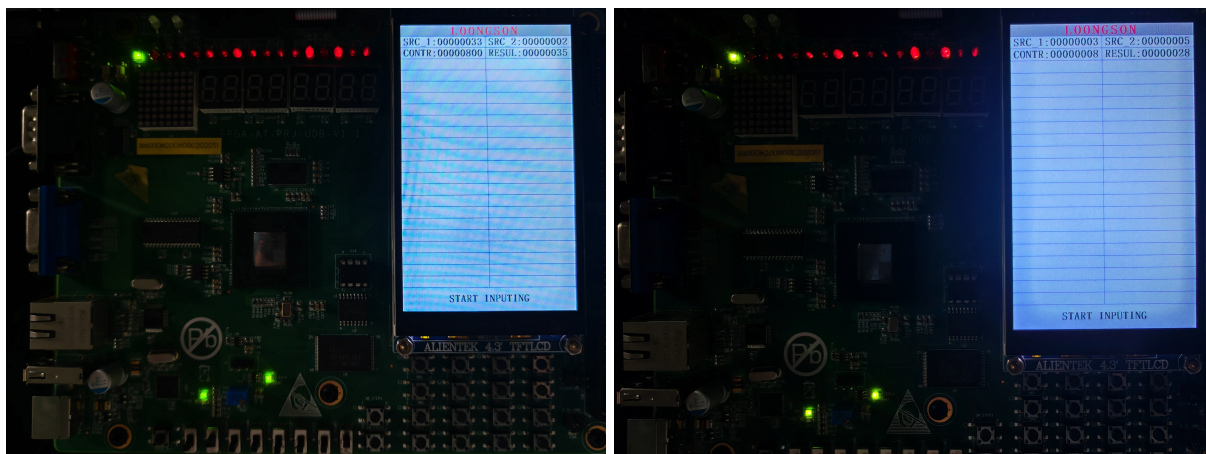


图 3: 实验箱运行结果

CONTR = 0x800 时 (即 100 000 000 000), 表示加法, $0x33+0x2 = 0x35$, 计算结果正确; CONTR = 0x8 时 (即 1000), 表示逻辑左移, $0x5$ 左移 3 位, 得到结果 $0x28$ (40), 计算结果正确。

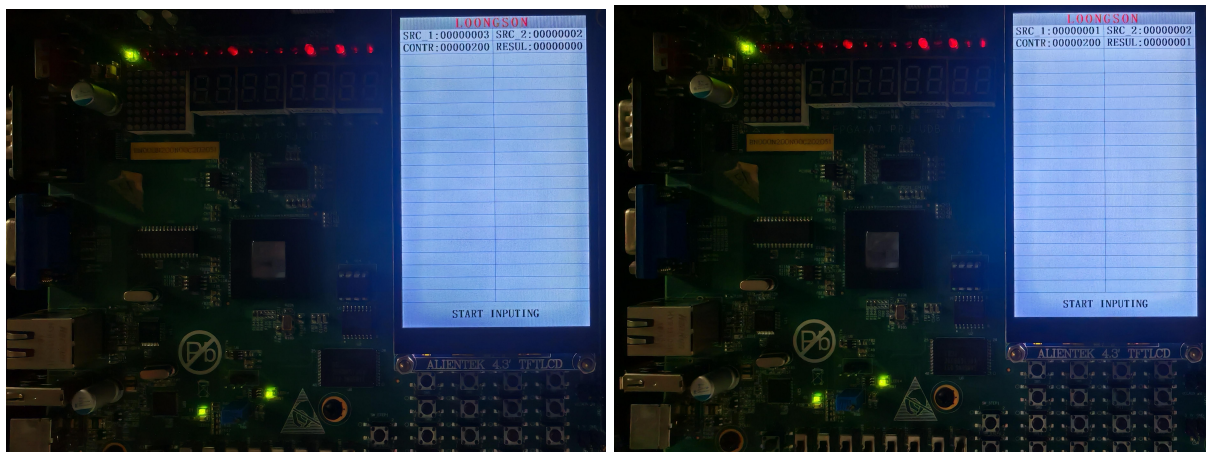


图 4: 实验箱运行结果

CONTR = 0x200 时 (即 1 000 000 000), 表示有符号比较, 图中 $0x3 > 0x2$ 输出 0, $0x1 < 0x2$ 输出 1, 计算结果正确。

篇幅原因此处仅展示部分的测试,经过实际测试,各项计算均准确无误。接下来将以此为基础,依照实验要求对其进行改进与操作扩展。

2.ALU 扩展

实现了基础的 ALU 后,依据实验要求,对其进行功能改进与操作扩展。在这部分,主要需要修改核心的寄存器模块 alu.v 与顶层的展示模块 alu_display.v,然后对约束文件进行相应的修改,对照引脚编号调整配置。

i. 设计文件

首先在核心模块 alu.v 中将控制信号改为 4 位,再在模块中添加扩展的几种运算及其相应的计算逻辑:

```
1  `timescale 1ns / 1ps
2
3  module alu(
4      input  [3:0]  alu_control,  // 修改为4位ALU控制信号
5      input  [31:0] alu_src1,      // ALU操作数1,为补码
6      input  [31:0] alu_src2,      // ALU操作数2,为补码
7      output [31:0] alu_result     // ALU结果
8  );
9
10     // ALU控制信号,改为4位二进制译码
11     wire alu_add  = (alu_control == 4'b0000);
12     wire alu_sub  = (alu_control == 4'b0001);
13     wire alu_slt  = (alu_control == 4'b0010);
14     wire alu_sltu = (alu_control == 4'b0011);
15     wire alu_and  = (alu_control == 4'b0100);
16     wire alu_nor  = (alu_control == 4'b0101);
17     wire alu_or   = (alu_control == 4'b0110);
18     wire alu_xor  = (alu_control == 4'b0111);
19     wire alu_sll  = (alu_control == 4'b1000);
20     wire alu_srl  = (alu_control == 4'b1001);
21     wire alu_sra  = (alu_control == 4'b1010);
22     wire alu_lui  = (alu_control == 4'b1011);
23
24     // 扩展新增的四种运算
25     wire alu_seq  = (alu_control == 4'b1100);  // 判断相等 SEQ
26     wire alu_nand = (alu_control == 4'b1101);  // 按位与非 NAND
27     wire alu_lll  = (alu_control == 4'b1110);  // 低位加载 LLI
28     wire alu_xnor = (alu_control == 4'b1111);  // 按位同或 XNOR
29
30     wire [31:0] add_sub_result;
31     wire [31:0] slt_result;
32     wire [31:0] sltu_result;
33     wire [31:0] and_result;
34     wire [31:0] nor_result;
35     wire [31:0] or_result;
36     wire [31:0] xor_result;
37     wire [31:0] sll_result;
38     wire [31:0] srl_result;
39     wire [31:0] sra_result;
```

```

40     wire [31:0] lui_result;
41
42     // 新增连线
43     wire [31:0] seq_result;
44     wire [31:0] nand_result;
45     wire [31:0] lli_result;
46     wire [31:0] xnor_result;
47
48     assign and_result = alu_src1 & alu_src2;           // 与结果为两数按位与
49     assign or_result  = alu_src1 | alu_src2;           // 或结果为两数按位或
50     assign nor_result = ~or_result;                   // 或非结果为或结果按位取反
51     assign xor_result = alu_src1 ^ alu_src2;           // 异或结果为两数按位异或
52     assign lui_result = {alu_src2[15:0], 16'd0};       // 立即数装载结果为立即数移位至高
        半字节
53
54     // 新增的运算逻辑
55     assign seq_result  = (alu_src1 == alu_src2) ? 32'd1 : 32'd0; // 相等置1, 否则0
56     assign nand_result = ~and_result; // 与非
57     assign lli_result  = {16'd0, alu_src2[15:0]}; // 低位装载立即数
58     assign xnor_result = ~xor_result; // 同或
59
60     // 此处未做更改, 省略代码原文
61     // .....
62
63     // 选择相应结果输出
64     // 加入新增指令
65     assign alu_result = (alu_add|alu_sub) ? add_sub_result[31:0] :
66                                     alu_slt      ? slt_result :
67                                     alu_sltu     ? sltu_result :
68                                     alu_and       ? and_result :
69                                     alu_nor       ? nor_result :
70                                     alu_or        ? or_result  :
71                                     alu_xor       ? xor_result :
72                                     alu_sll       ? sll_result :
73                                     alu_srl       ? srl_result :
74                                     alu_sra       ? sra_result :
75                                     alu_lui       ? lui_result :
76                                     alu_seq       ? seq_result : // 判断相等
77                                     alu_nand      ? nand_result: // 与非
78                                     alu_lli       ? lli_result : // 低位加载
79                                     alu_xnor      ? xnor_result: // 同或
80                                     32'd0;
81 endmodule

```

依照实验要求, 首先将控制信号改为 4 位二进制译码的形式, 再添加四种运算, 配置控制码并实现相应的计算逻辑, 最后将其连接至对应的输出即可。

经过以上对模块的修改, 已经可以满足实验要求的功能逻辑, 接下来根据 ALU 新增的功能在顶层模块 alu_display.v 上进行相应的改动, 配置相关的控制与显示等功能。

```

1
2 // .....
3
4 //-----{调用ALU模块}begin
5     wire [3:0] alu_control = alu_op_sw; // 直接使用物理拨码开关的值作为控制码
6     reg [31:0] alu_src1; // ALU操作数1
7     reg [31:0] alu_src2; // ALU操作数2
8     wire [31:0] alu_result; // ALU结果
9
10    alu alu_module(
11        .alu_control(alu_control),
12        .alu_src1 (alu_src1 ),
13        .alu_src2 (alu_src2 ),
14        .alu_result (alu_result )
15    );
16 //-----{调用ALU模块}end
17
18 // .....
19
20 //-----{输出到触摸屏显示}begin
21     always @(posedge clk)
22     begin
23         case(display_number)
24             6'd1 :
25                 begin
26                     display_valid <= 1'b1;
27                     display_name <= "SRC_1";
28                     display_value <= alu_src1;
29                 end
30             6'd2 :
31                 begin
32                     display_valid <= 1'b1;
33                     display_name <= "SRC_2";
34                     display_value <= alu_src2;
35                 end
36             6'd3 :
37                 begin
38                     display_valid <= 1'b1;
39                     display_name <= "CONTR";
40                     // 修改：控制码变成了4位，需要补足前28位0进行显示
41                     display_value <= {28'd0, alu_control};
42                 end
43             6'd4 :
44                 begin
45                     display_valid <= 1'b1;
46                     display_name <= "RESUL";
47                     display_value <= alu_result;
48                 end
49             default :

```

```

50         begin
51             display_valid <= 1'b0;
52             display_name  <= 40'd0;
53             display_value <= 32'd0;
54         end
55     endcase
56 end
57 //-----{输出到触摸屏显示}
58
59 // .....

```

在这部分，先把写 alu_control 改为使用拨码开关的值进行控制，这是控制码变为了 4 位，为了使其可以再 LCD 屏幕上正常显示，在输出时还需补足前 28 位 0。经过以上改动，模块的功能已经较为完善，接下来将对其进行功能验证。

ii. 仿真文件

```

1  `timescale 1ns / 1ps
2
3  module alu_tb;
4      reg [3:0]  alu_control;
5      reg [31:0] alu_src1;
6      reg [31:0] alu_src2;
7
8      wire [31:0] alu_result;
9
10     // ALU 模块
11     alu uut (
12         .alu_control(alu_control),
13         .alu_src1(alu_src1),
14         .alu_src2(alu_src2),
15         .alu_result(alu_result)
16     );
17
18     initial begin
19         // 初始化
20         alu_control = 4'b0000;
21         alu_src1 = 32'd0;
22         alu_src2 = 32'd0;
23         #50;
24         // 1. 加法 (ADD)
25         alu_control = 4'b0000; alu_src1 = 32'h12345678; alu_src2 = 32'h87654321;
26         #20;
27         $display("加法 (ADD) | CONTR: %b | SRC_1: %h | SRC_2: %h | RESUL: %h",
28             alu_control, alu_src1, alu_src2, alu_result);
29
30         // 2. 减法 (SUB)
31         alu_control = 4'b0001; alu_src1 = 32'h00000010; alu_src2 = 32'h00000020;
32         #20;

```

```

30     $display("减法 (SUB) | CONTR: %b | SRC_1: %h | SRC_2: %h | RESUL: %h",
31             alu_control, alu_src1, alu_src2, alu_result);
32
33     // 3. 有符号比较 (SLT)
34     alu_control = 4'b0010; alu_src1 = 32'hFFFFFFF; alu_src2 = 32'h00000001;
35     #20;
36     $display("比较 (SLT) | CONTR: %b | SRC_1: %h | SRC_2: %h | RESUL: %h",
37             alu_control, alu_src1, alu_src2, alu_result);
38
39     // 4. 按位与 (AND)
40     alu_control = 4'b0100; alu_src1 = 32'h0F0F0F0F; alu_src2 = 32'h0000FFFF;
41     #20;
42     $display("与门 (AND) | CONTR: %b | SRC_1: %h | SRC_2: %h | RESUL: %h",
43             alu_control, alu_src1, alu_src2, alu_result);
44
45     // 5. 逻辑左移 (SLL)
46     alu_control = 4'b1000; alu_src1 = 32'h00000004; alu_src2 = 32'h12345678;
47     #20;
48     $display("左移 (SLL) | CONTR: %b | SRC_1: %h | SRC_2: %h | RESUL: %h",
49             alu_control, alu_src1, alu_src2, alu_result);
50
51     // 6. 相等比较 (SEQ - 相同情况)
52     alu_control = 4'b1100; alu_src1 = 32'hAABBCCDD; alu_src2 = 32'hAABBCCDD;
53     #20;
54     $display("相等 (SEQ) | CONTR: %b | SRC_1: %h | SRC_2: %h | RESUL: %h",
55             alu_control, alu_src1, alu_src2, alu_result);
56
57     // 7. 相等比较 (SEQ - 相异情况)
58     alu_control = 4'b1100; alu_src1 = 32'hAABBCCDD; alu_src2 = 32'h11223344;
59     #20;
60     $display("相等 (SEQ) | CONTR: %b | SRC_1: %h | SRC_2: %h | RESUL: %h",
61             alu_control, alu_src1, alu_src2, alu_result);
62
63     // 8. 按位与非 (NAND)
64     alu_control = 4'b1101; alu_src1 = 32'h0000FFFF; alu_src2 = 32'h0F0F0F0F;
65     #20;
66     $display("与非 (NAND) | CONTR: %b | SRC_1: %h | SRC_2: %h | RESUL: %h",
67             alu_control, alu_src1, alu_src2, alu_result);
68
69     // 9. 低位加载 (LLI)
70     alu_control = 4'b1110; alu_src1 = 32'h12345678; alu_src2 = 32'hDEADBEEF;
71     #20;
72     $display("加载 (LLI) | CONTR: %b | SRC_1: %h | SRC_2: %h | RESUL: %h",
73             alu_control, alu_src1, alu_src2, alu_result);
74
75     // 10. 按位同或 (XNOR - 相同情况)
76     alu_control = 4'b1111; alu_src1 = 32'h12345678; alu_src2 = 32'h12345678;
77     #20;
78     $display("同或 (XNOR) | CONTR: %b | SRC_1: %h | SRC_2: %h | RESUL: %h",
79             alu_control, alu_src1, alu_src2, alu_result);

```

```

63
64 // 11. 按位同或 (XNOR - 相异情况)
65 alu_control = 4'b1111; alu_src1 = 32'h00000000; alu_src2 = 32'hFFFFFFFF;
66     #20;
67 $display("同或 (XNOR) | CONTR: %b | SRC_1: %h | SRC_2: %h | RESUL: %h",
68     alu_control, alu_src1, alu_src2, alu_result);
69
70 $finish;
71 end
endmodule

```

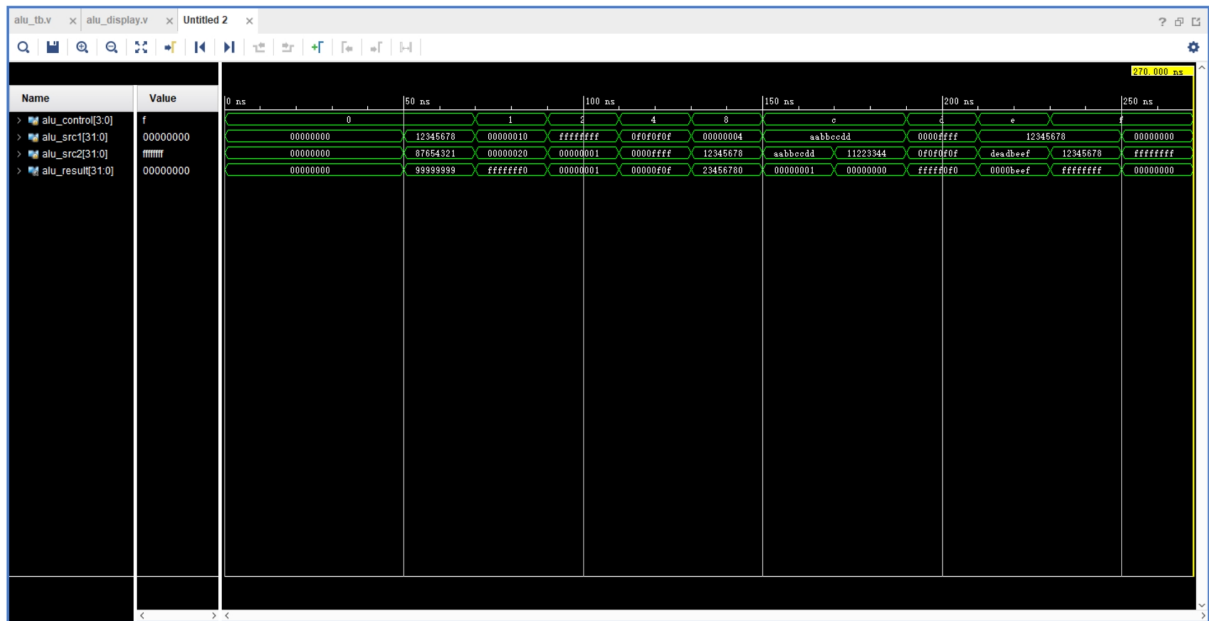


图 5: 仿真波形图

为了能更清晰地查看数据，此处将测试样例的详细信息都打印在控制台上，得到以下结果：

```

1 加法 (ADD) | CONTR: 0000 | SRC_1: 12345678 | SRC_2: 87654321 | RESUL: 99999999
2 减法 (SUB) | CONTR: 0001 | SRC_1: 00000010 | SRC_2: 00000020 | RESUL: ffffffff
3 比较 (SLT) | CONTR: 0010 | SRC_1: ffffffff | SRC_2: 00000001 | RESUL: 00000001
4 与门 (AND) | CONTR: 0100 | SRC_1: 0f0f0f0f | SRC_2: 0000ffff | RESUL: 0000f0f
5 左移 (SLL) | CONTR: 1000 | SRC_1: 00000004 | SRC_2: 12345678 | RESUL: 23456780
6 相等 (SEQ) | CONTR: 1100 | SRC_1: aabbccdd | SRC_2: aabbccdd | RESUL: 00000001
7 相等 (SEQ) | CONTR: 1100 | SRC_1: aabbccdd | SRC_2: 11223344 | RESUL: 00000000
8 与非 (NAND) | CONTR: 1101 | SRC_1: 0000ffff | SRC_2: 0f0f0f0f | RESUL: fffff0f
9 加载 (LLI) | CONTR: 1110 | SRC_1: 12345678 | SRC_2: deadbeef | RESUL: 0000beef
10 同或 (XNOR) | CONTR: 1111 | SRC_1: 12345678 | SRC_2: 12345678 | RESUL: ffffffff
11 同或 (XNOR) | CONTR: 1111 | SRC_1: 00000000 | SRC_2: ffffffff | RESUL: 00000000

```

经过验证，上述计算结果均准确无误，接下来，将对约束文件进行相应的修改，使其可以在实验箱上运行与测试。

iii. 约束文件

这里使用 S2 到 S5 这 4 个拨码开关作为 alu 的控制码，在 alu.xdc 中添加以下代码：

```
1 # SW2 -> alu_op_sw[0], SW3 -> alu_op_sw[1], SW4 -> alu_op_sw[2], SW5 -> alu_op_sw
   [3]
2 set_property PACKAGE_PIN AC22 [get_ports {alu_op_sw[0]}]
3 set_property PACKAGE_PIN AC23 [get_ports {alu_op_sw[1]}]
4 set_property PACKAGE_PIN AB6 [get_ports {alu_op_sw[2]}]
5 set_property PACKAGE_PIN W6 [get_ports {alu_op_sw[3]}]
6
7 set_property IOSTANDARD LVCMOS33 [get_ports {alu_op_sw[0]}]
8 set_property IOSTANDARD LVCMOS33 [get_ports {alu_op_sw[1]}]
9 set_property IOSTANDARD LVCMOS33 [get_ports {alu_op_sw[2]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {alu_op_sw[3]}]
```

iv. 结果验证

代码修改完成后，逐步生成位流文件并烧录至实验箱，在实验箱上进行功能验证。
调节拨码开关，设置输入值与控制码，进行如下测试：

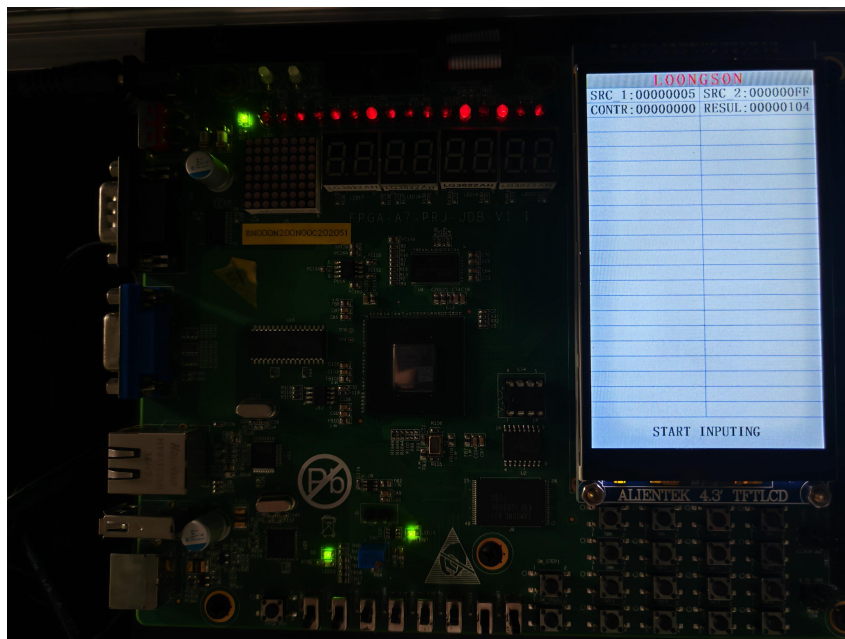


图 6: 实验箱运行结果

CONTR = 0x0 时，表示加法， $0x5 + 0xFF = 0x104$ ，计算结果正确。

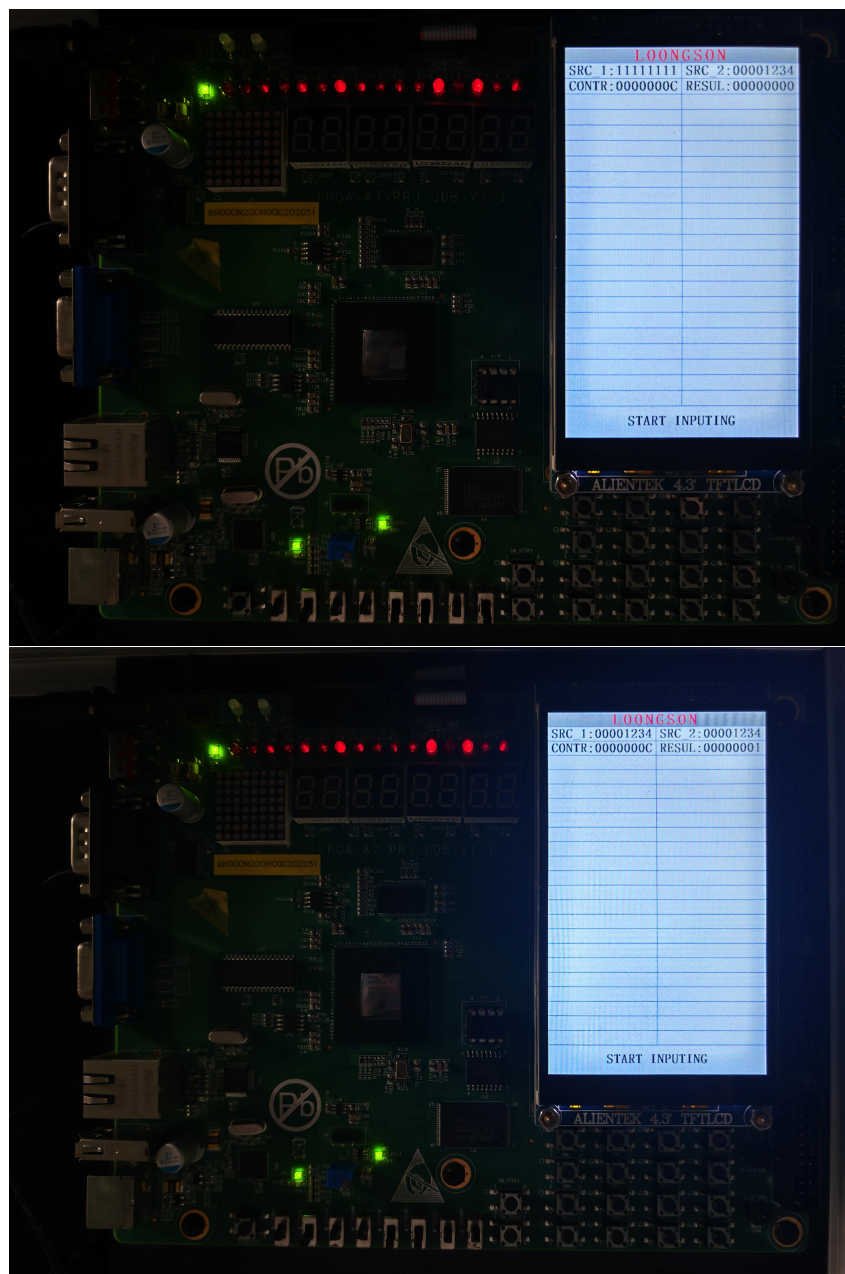


图 7: 实验箱运行结果

CONTR = 0xC 时，表示判断相等， $0x11111111 \neq 0x1234$ 输出 0； $0x1234 == 0x1234$ 输出 1，计算结果正确。

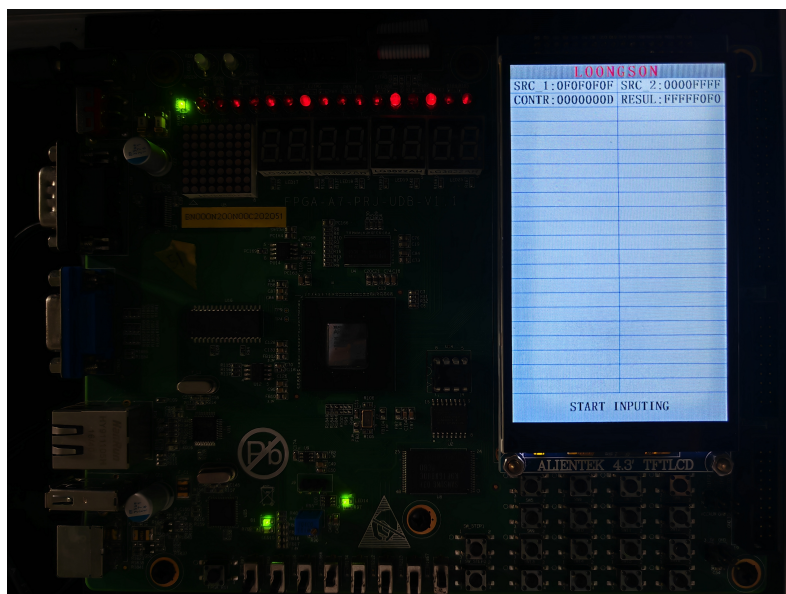


图 8: 实验箱运行结果

CONTR = 0xD 时，表示按位与非，SRC1 = 0x0F0F0F0F，SRC2 = 0x0000FFFF 经过与非运算得到 0xFFFFF0F0，计算结果正确。

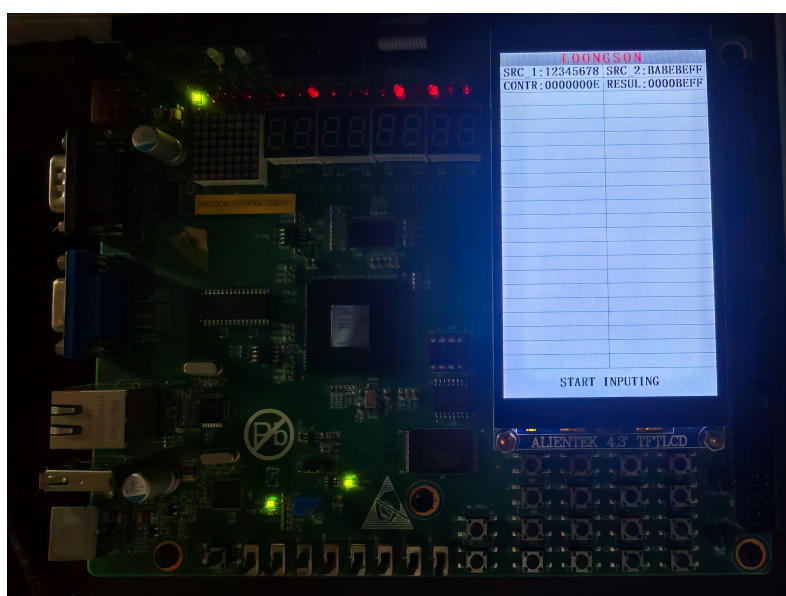


图 9: 实验箱运行结果

CONTR = 0xE 时，表示低位加载，取 SRC2 = 0xBABEBEEF 的低位，输出 0x0000BEEF，计算结果正确。

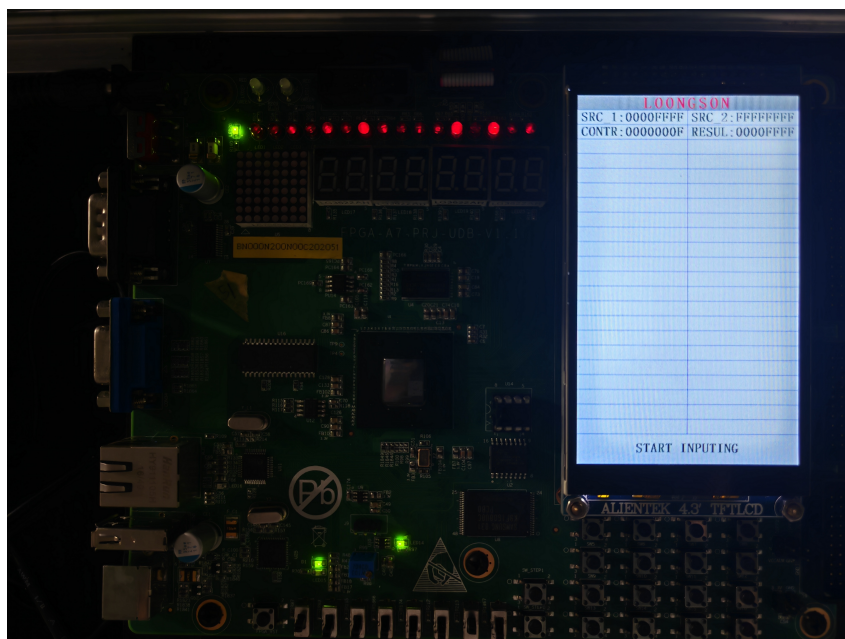


图 10: 实验箱运行结果

CONTR = 0xF 时，表示按位同或，SRC1 = 0x0000FFFF，SRC2 = 0xFFFFFFFF 经过同或运算得到 0x0000FFFF，计算结果正确。

综合上述验证结果，实验箱各项功能均准确无误，完美符合实验预期，验证了代码逻辑的正确性。

五、总结感想

本次实验中，首先通过提供的源码实现了基础的 ALU。了解原理后，在此基础上进行功能改进，将控制码由 12 位独热码改为使用 4 位拨码开关与译码器来控制，同时还新增 4 种运算，依照要求完成了对于 ALU 的功能改进与扩展。

通过这次实验，我了解了 ALU 的原理和代码实现逻辑，在实践中对于 ALU 的基础结构有了进一步的理解。这一元件最核心的功能就是实现各种运算，再通过控制码来确定不同的输出。在具体实现中，运算的实现逻辑是最为重要的部分，这些实践的过程将计算机的原理和逻辑代数结合起来，让我对于硬件的本质有了更加深入的认知；同时，在实验箱上进行相关的功能验证时，调试的一系列过程也体现出工程实践的重要性，让我强化了代码逻辑和现实硬件之间的关联思维。